

# Milestone 4 Report

Smart MCQ Solver Challenge — Multiple-Choice Fine-Tuning with BERT and LoRA

This milestone covers the full pipeline for fine-tuning a multiple-choice model: MCQ data formatting, tokenization into the [batch, choices, seq\_len] shape, using `AutoModelForMultipleChoice`, applying LoRA adapters for parameter-efficient fine-tuning, preparing data for the Hugging Face Trainer, running a small training job, and running inference to get per-option probabilities.

## Section 1 — MCQ Data Formatting (Q1–Q2)

### Q1. Label encoding for row 150

Mapped the answer column using A=0, B=1, C=2, D=3, E=4 with pandas `.map()`. The question asks for the numeric label at index 150, which is whatever letter is in the answer column for that row translated through this mapping.

↳ Numeric labels are required by PyTorch's `CrossEntropyLoss`, which `AutoModelForMultipleChoice` uses internally. String letters can't be passed directly as targets.

### Q2. Character length of Option B formatted string for row 0

Built the formatted string as: `str(prompt) + " [SEP] " + str(option_B)` exactly as specified. Measured character length with Python's `len()`. The [SEP] separator with spaces on both sides signals to BERT where the first sentence (prompt) ends and the second (the option being evaluated) begins.

↳ This is the standard BERT sentence-pair format. BERT was pre-trained on sentence pairs separated by [SEP], so using this format for prompt+option pairs aligns with the pre-training setup and generally improves downstream performance on classification tasks.

## Section 2 — Tokenization for Multiple-Choice Models (Q3–Q4)

### Q3. Second dimension of the [1, 5, 128] input\_ids tensor

Tokenized all 5 formatted inputs for row 0 together (passing a list of 5 strings to the tokenizer) with `padding='max_length', truncation=True, max_length=128, return_tensors='pt'`. This produces a [5, 128] tensor. Added a batch dimension with `.unsqueeze(0)` to get [1, 5, 128]. The second dimension is 5 — the number of answer choices.

↳ `AutoModelForMultipleChoice` expects shape [batch\_size, num\_choices, seq\_length]. The model runs BERT on each of the 5 (prompt, option) pairs independently, then compares their [CLS] representations through a linear layer to produce 5 logit scores.

### Q4. Total token positions in the [16, 5, 128] batch tensor

Tokenized each of the first 16 rows separately (each producing [5, 128]), then stacked with `torch.stack()` to get [16, 5, 128]. Total token positions =  $16 \times 5 \times 128 = 10,240$ . Used `.numel()` to confirm.

↳ Each of those 10,240 positions holds an integer token ID. The corresponding `attention_mask` tensor has the same shape and marks which positions are real tokens (1) vs. padding (0).

## Section 3 — Multiple-Choice Model Outputs (Q5–Q6)

### Q5. Number of logits produced for one question

Loaded `bert-base-uncased` with `AutoModelForMultipleChoice`, passed the [1, 5, 128] `input_ids` and `attention_mask` for row 0. The output logits tensor has shape [1, 5] — one score per answer choice. The answer is 5.

↳ `AutoModelForMultipleChoice` adds a linear classification head on top of BERT's [CLS] token output. For each of the 5 choices, it independently runs the full BERT stack, extracts the [CLS] embedding, and projects it to a single scalar. These 5 scalars are the logits.

## Q6. Number of dimensions of the scalar loss tensor

Passed the tokenized row 0 inputs plus the correct label (as a [1]-shaped tensor) to `AutoModelForMultipleChoice`. When labels are provided, the model computes `CrossEntropyLoss` internally and returns it in `outputs.loss`. A scalar loss is a 0-dimensional tensor in PyTorch (`tensor.ndim == 0`). The answer is 0.

↳ A 0-dim tensor is different from a 1-dim tensor of length 1. `torch.tensor(2.5)` has `ndim=0`; `torch.tensor([2.5])` has `ndim=1`. The loss is always 0-dim because it's a single number, not a batch of numbers.

## Section 4 — LoRA for Efficient Fine-Tuning (Q7)

### Q7. Number of trainable parameters after applying LoRA

Applied `LoraConfig` with `r=8`, `lora_alpha=16`, `target_modules=["query","value"]`, `lora_dropout=0.1`, `bias="none"`, `task_type=TaskType.SEQ_CLS` via `get_peft_model()`. LoRA freezes all original BERT weights and adds small rank-8 decomposition matrices (A and B) to the query and value projection layers only. Trainable parameters = sum of numel for params with `requires_grad=True`.

↳ With `r=8` and targeting query+value in all 12 BERT layers, LoRA adds 4 matrices per layer (`query_A`, `query_B`, `value_A`, `value_B`). Each is shaped (`hidden_dim`, `r`) or (`r`, `hidden_dim`), giving  $768 \times 8 + 8 \times 768 = 12,288$  params per matrix  $\times$  4 matrices  $\times$  12 layers = 589,824 adapter params, plus any bias/classification head that wasn't frozen.

## Section 5 — HuggingFace Dataset Preparation (Q8)

### Q8. Number of tokenized choices stored in `input_ids` for the first dataset item

Tokenized all 100 rows using the same `format_choices()` function (prompt [SEP] option), with `max_length=128`, and stored each as a dict with keys `input_ids` (list of 5 lists of 128 ints), `attention_mask` (same shape), and `label` (int). Wrapped in `Dataset.from_list()`. The first item's `input_ids` has 5 inner lists — one per choice. The answer is 5.

↳ The Trainer expects a 'labels' key (not 'label') when computing loss automatically. The dataset uses 'label' here for inspection, but the training dataset in Q9 uses 'labels' to match what Trainer expects.

## Section 6 — Tiny Fine-Tuning and Inference (Q9–Q10)

### Q9. Final `global_step` reported by the Trainer

Created a fresh LoRA model, tokenized the first 32 rows with `max_length=64`, built a Dataset with the 'labels' key (required by Trainer), and ran Trainer with `max_steps=4`, `per_device_train_batch_size=4`, `gradient_accumulation_steps=1`. With 32 rows, batch size 4, and a hard cap of `max_steps=4`, the Trainer stops after exactly 4 gradient steps. `global_step = 4`.

↳ `max_steps` overrides `num_train_epochs` — it's an absolute step count ceiling. With `batch_size=4` and 32 rows, one full epoch would be 8 steps, but `max_steps=4` stops it at exactly half an epoch. This is a deliberate tiny training run just to verify the pipeline works end-to-end.

### Q10. Probability assigned to option E after fine-tuning

Re-ran inference on row 0 with the fine-tuned LoRA model, using `max_length=64` (matching the training tokenization). Applied `F.softmax(logits, dim=-1)` to convert the 5 raw logits to probabilities summing to 1.0. Extracted index 4 (option E) and rounded to 4 decimal places.

↳ The model was only trained for 4 steps on 32 rows, so the probabilities reflect a model that's barely started training — mostly still the pre-trained BERT initialization. The softmax values will be close to uniform (near 0.2 each) unless one option happened to be strongly preferred by the un-fine-tuned model.

*The overall shape of Milestone 4 is: format data as sentence pairs → tokenize into [batch, 5, 128] tensors → AutoModelForMultipleChoice produces [batch, 5] logits → LoRA adapters make fine-tuning feasible on a single GPU → Trainer handles the training loop → softmax converts logits to per-option probabilities for inference. Each step has a specific shape or dimension that the next step depends on, which is why the questions focus on getting those shapes right.*